

Clase 1: Introducción a No-SQL

Bases de Datos Documentales y de Clave - Valor

Parte 1

El Modelo Relacional y la Impedancia

El Modelo Relacional

Estructura Tradicional

Comencemos explorando el modelo relacional clásico, el cual organiza la información en una estructura estricta de **tablas y filas** (o relaciones y tuplas).

- » Los valores en una tupla relacional deben ser **simples**.
- » No pueden contener estructuras ricas o complejas como nested records o listas.
- » Requiere **esquemas rígidos** predefinidos antes de la inserción de datos.

Customers	
customerID	Int
firstName	String
lastName	String
birthDate	Date
moneySpent	Money
anniversary	Date

Products	
productID	Int
category	String
price	Money

Employees	
employeeID	Int
firstName	String
lastName	String
birthDate	Date

Orders	
orderID	Int
customerID	Int
employeeID	Int
productID	Int
orderTotal	Money
orderDate	Date

El Problema de la Impedancia

Discrepancia Estructural

El "impedance mismatch" se refiere a la gran diferencia entre los modelos de datos relacionales simples y las **estructuras de datos ricas en memoria** (como OOP) utilizadas por los desarrolladores modernos.

Un único objeto lógico en una aplicación a menudo tiene que ser dividido y distribuido en múltiples filas y tablas relacionales, requiriendo traducción constante.

Costos de Productividad

Se desperdicia una cantidad significativa de esfuerzo en el desarrollo de aplicaciones simplemente mapeando datos entre estructuras en memoria y bases de datos relacionales (ORMs). Esto incrementa la complejidad del código, reduce el rendimiento general y enlentece la entrega de nuevas funcionalidades de negocio.

Ejemplo: El Desajuste en la Práctica

En el Código (Objetos)

Lenguajes como Java o Python manejan datos de forma **jerárquica y natural**:

```
class Student {  
  int id;  
  String name;  
  Address address; // Anidado  
}
```

En la BD (Relacional)

Las bases relacionales "**aplanan**" la estructura en tablas vinculadas:

Tabla: Student

id | name | addr_id

Tabla: Address

id | street | city



Consecuencia: Esta diferencia requiere costosas operaciones **JOIN** y el uso de herramientas **ORM** (Hibernate, SQLAlchemy) para actuar como puente, aumentando la complejidad del sistema.

No hay fragmentación estructural en el código; el objeto "contiene" toda la información en memoria.

OOP vs RDBMS

Feature	Relational Database (RDBMS)	Object-Oriented Database (OODBMS)
Data Model	Tables with rows and columns (relational model).	Objects, supporting encapsulation, inheritance, and polymorphism.
Query Language	Uses the standardized SQL (Structured Query Language).	Often uses proprietary query methods or direct programming language integration (e.g., ObjectDB for Java).
Data Handling	Excellent for structured data and complex, ad-hoc queries with strong consistency and transactions (ACID compliant).	Naturally handles complex data structures, multimedia content, and intricate relationships (graph-like structures).
Integration	Requires a translation layer (Object-Relational Mapper or ORM, like Hibernate for Java or SQLAlchemy for Python) to work with OOP languages, leading to an "impedance mismatch".	Provides seamless integration with OOP languages, eliminating the impedance mismatch and allowing developers to work directly with objects.
Popularity/Ecosystem	Widely adopted, mature technology with extensive tools, community support, and developer familiarity.	Less popular, with limited tools, community support, and a steeper learning curve.

Parte 2

Escalabilidad, Sistemas Distribuidos y NoSQL

Scaling Up vs. Scaling Out

Scaling Up (Vertical)

Consiste en comprar máquinas más grandes, más potentes y cada vez más costosas para soportar el crecimiento de datos (terabytes).

- » Aumentos de CPU, RAM o Discos en un solo servidor.
- » Altos costos financieros.
- » **Límite físico:** Eventualmente se alcanza un límite donde no se puede comprar una máquina más potente a ningún precio.

Scaling Out (Horizontal)

Consiste en particionar los datos a través de clústeres de servidores commodity (hardware comercial estándar más económico).

- » Se añaden más nodos al clúster según la demanda.
- » Capacidad de crecimiento virtualmente infinita.
- » Requiere que el software de base de datos soporte distribución nativa.

Limitaciones Relacionales en Clústeres

El Desafío de la Distribución

Las bases de datos relacionales fueron diseñadas principalmente para ejecutarse en **máquinas individuales**.

Intentar ejecutarlas en clústeres a menudo resulta en:

- » **Single points of failure**, como subsistemas de discos compartidos.
- » Requerimientos de **Sharding manual** ("unnatural acts").
- » Pérdida de consultas entre shards (cross-shard querying).
- » Pérdida de la integridad referencial.



El Movimiento NoSQL y el Teorema CAP



Nacido para Clústeres

Empresas como Google y Amazon impulsaron la necesidad de bases de datos diseñadas explícitamente para vivir en un mundo de clústeres distribuidos desde su concepción.



Consistencia Variable

Operar en clústeres requiere diferentes modelos de distribución. Las transacciones ACID tradicionales chocan inherentemente con los entornos de clúster escalables.



Teorema CAP de Brewer

En sistemas distribuidos, solo se pueden garantizar dos de tres: **C**onsistency, **A**vailability), y **P**artition Tolerance. NoSQL flexibiliza esto.

¿Qué es NoSQL? (Not Only SQL)

Sistemas de datos no relacionales y distribuidos diseñados para ofrecer **flexibilidad, escalabilidad horizontal y alto rendimiento**. A diferencia de las tablas rígidas de SQL, utilizan modelos variados para manejar datos no estructurados.

Flexible or Schema-less

Permiten estructuras dinámicas donde se pueden añadir campos sobre la marcha sin actualizar un esquema rígido.

Horizontal Scaling

Crecen añadiendo más servidores a un clúster en lugar de depender de un único servidor costoso.

High Performance

Optimizados para modelos de datos y patrones de acceso específicos, acelerando las operaciones de lectura/escritura.

Distributed Architecture

Nacidos para el cloud computing, ofreciendo alta disponibilidad y tolerancia a fallos de forma nativa.

¿Qué es NoSQL? (Not Only SQL)

Sistemas de datos no relacionales y distribuidos diseñados para ofrecer **flexibilidad, escalabilidad horizontal y alto rendimiento**. A diferencia de las tablas rígidas de SQL, utilizan modelos variados para manejar datos no estructurados.

Flexible or Schema-less

Permiten estructuras dinámicas donde se pueden añadir campos sobre la marcha sin actualizar un esquema rígido.

Horizontal Scaling

Crecen añadiendo más servidores a un clúster en lugar de depender de un único servidor costoso.

High Performance

Optimizados para modelos de datos y patrones de acceso específicos, acelerando las operaciones de lectura/escritura.

Distributed Architecture

Nacidos para el cloud computing, ofreciendo alta disponibilidad y tolerancia a fallos de forma nativa.

Teorema CAP y la Flexibilidad NoSQL

triangle

El Dilema del Triángulo

Eric Brewer establece que en sistemas distribuidos solo puedes elegir **dos de tres**:

Consistencia (Consistency)

Availability (Disponibilidad)

Partition Tolerance (Tolerancia a Particiones)

tune

Parámetros Ajustables (R, W, N)

Bases de datos como **Riak** permiten ajustar el sistema según la necesidad:

N: Factor de replicación.

R/W: Nodos requeridos para lectura/escritura exitosa.

Ej: R=5, W=5 prioriza consistencia; R=1, W=1 prioriza disponibilidad.

psycholog
y

La Nueva Responsabilidad del Desarrollador

Adiós al "Default" RDBMS

Ya no hay un estándar fijo de transacciones ACID. Debemos evaluar si cada operación es crítica o tolera pérdida.

Análisis de Carga

¿El sistema es pesado en lectura o escritura? ¿Requiere consultas aleatorias? El diseño depende totalmente del uso.

Herramientas Externas

Para necesidades específicas de coordinación,.

Comparativa: Bases Relacionales vs NoSQL

Característica	storage Relacional (SQL)	grid_view NoSQL
Esquema (Schema)	cancel Rígido / Estático Requiere definir columnas y tipos antes de insertar. Difícil de alterar.	check_circle Flexible / Dinámico Datos no estructurados sobre la marcha. Ideal para iteraciones rápidas.
Escalabilidad	arrow_upward Vertical (Scale-Up) Servidores más caros (RAM, CPU). Tiene un límite de crecimiento físico.	sync_alt Horizontal (Scale-Out) Particionamiento fácil añadiendo servidores económicos al clúster.
Consistencia	verified_user Cumplimiento ACID Estricta integridad. Ideal para sistemas financieros y contables.	speed Modelo BASE Consistencia eventual para favorecer disponibilidad y rendimiento.
Relaciones (JOINS)	hub Optimizado Excelente para relacionar múltiples tablas con consultas complejas.	layers_clear Desnormalizado Se debe embeber datos. Los JOINS consumen muchos recursos.

Parte 3

Tipos de bases de datos NoSQL

Bases de Datos de Documentos

SC

rip

tipo

n

¿Qué es un Documento?

Estructuras de datos **jerárquicas y autodescriptivas** (XML, JSON, BSON) que contienen mapas, colecciones y valores escalares.

Son similares entre sí pero **no requieren un esquema idéntico**, ofreciendo máxima flexibilidad.

Evolución Clave-Valor

Imagine un almacén clave-valor donde el **valor es examinable**. Esto permite consultas ricas e índices internos.

BDs Populares

MongoDB

CouchDB

OrientDB

RavenDB

Lotus Notes / Terrastore



Bases de Datos Clave-Valor

vp

n_ Simplicidad y Rendimiento

ke
y

El modelo NoSQL más simple: los datos se almacenan como un **blob (valor)** asociado a una **clave única**.

Ideal para accesos ultrarrápidos mediante clave primaria y **escalabilidad masiva**.

Operaciones Básicas

- **GET:** Recuperar valor
- **PUT:** Almacenar valor
- **DELETE:** Eliminar clave

BDs Populares

Redis

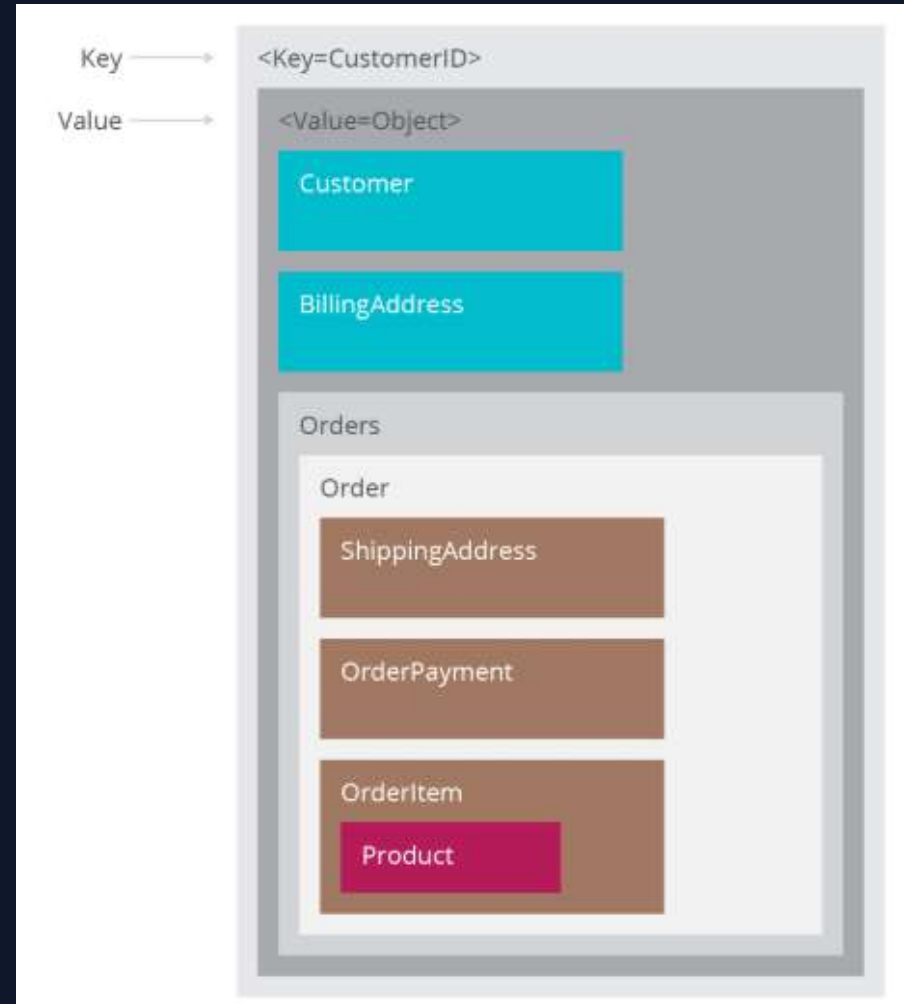
Riak

DynamoDB

Memcached

Couchbase

La persistencia varía: Memcached es volátil; Riak es persistente.



Bases de Datos de Familia de Columnas

¿Qué es una Familia de Columnas?

Almacenan datos en **grupos relacionados** que se acceden frecuentemente juntos. Cada fila se identifica por una **clave de fila** y puede tener un número variable de columnas.

A diferencia de un RDBMS, las filas **no requieren el mismo esquema**, permitiendo añadir columnas a cualquier fila en cualquier momento.

Super Columns

Cuando una columna consiste en un **mapa de columnas**, se denomina Super Columna. Actúa como un contenedor de columnas anidadas.

BDs Populares

Cassandra

HBase

DynamoDB

Hypertable

Cassandra destaca por su escalabilidad lineal y arquitectura sin maestro (p2p).



Bases de Datos de Grafos

hNodos y Relaciones

Almacenan **entidades (nodos)** y las **conexiones (relaciones)** entre ellas como ciudadanos de primera clase. A diferencia del RDBMS, las relaciones se **persisten físicamente**, no se calculan al momento de la consulta.

Permiten descubrir patrones complejos y realizar travesías profundas (traversals) con un rendimiento excepcionalmente alto.

Flexibilidad y Rapidez

- **Sin esquemas rígidos:** Añadir nuevos tipos de relaciones es instantáneo sin migraciones masivas.
- **Inteligencia:** Las relaciones tienen dirección y propiedades (ej: peso, tiempo, distancia) para consultas avanzadas.

BDs Populares

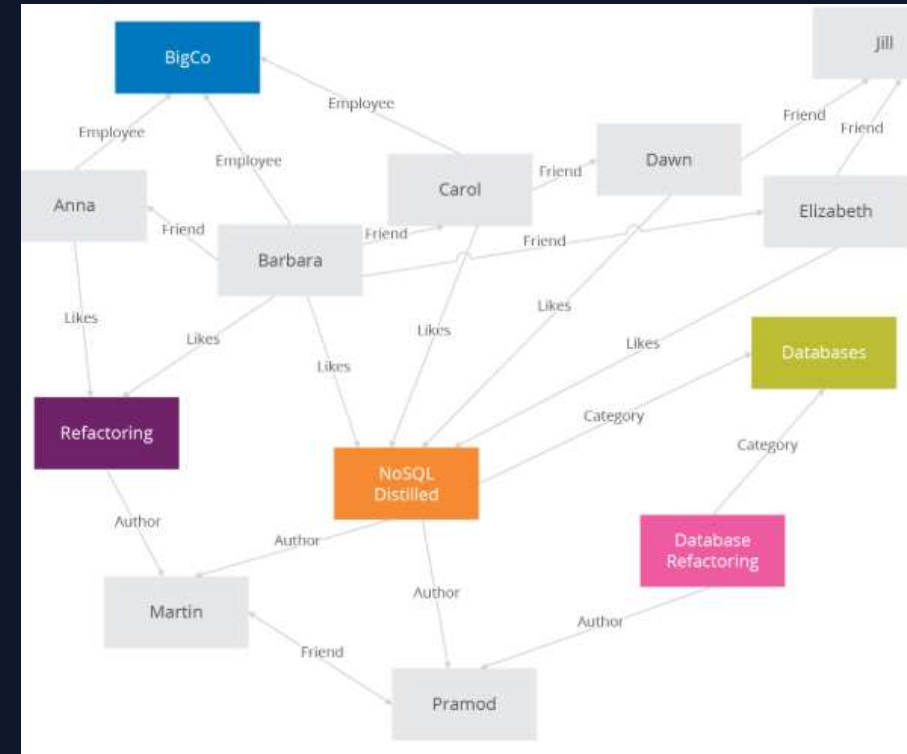
Neo4J

OrientDB

InfiniteGraph

FlockDB

FlockDB es un caso especial: solo soporta relaciones de profundidad simple (listas de adyacencia).



Parte 4

Showcase: Instalación y Configuración de MongoDB en
Windows

La Historia de MongoDB: De 10gen a Gigante NoSQL

groups 2007: El Origen

Dwight Merriman, Eliot Horowitz y Kevin Ryan (ex-DoubleClick) fundan **10gen** tras sufrir las limitaciones de las BD relacionales para escalar anuncios.

rocket_launch 2009: Lanzamiento

Lanzan **MongoDB** ("humongous"). Un modelo orientado a documentos diseñado para ser escalable, flexible y centrado en el desarrollador.

business 2013-2014: Expansión

La empresa se renombra a MongoDB. **Dev Ittycheria** asume como CEO en 2014, impulsando el crecimiento global y la innovación.

El Presente: MongoDB Atlas y Más Allá

- **MongoDB Atlas:** Una plataforma de datos unificada en la nube que integra múltiples servicios.
- **Inversión R&D:** Más de \$1B invertidos en investigación y desarrollo.
- **Impacto Global:** Cientos de millones de descargas y líder indiscutible para aplicaciones modernas.



El Enfoque Orientado a Documentos

Sustituyendo la Fila Clásica

MongoDB es una base de datos orientada a documentos que reemplaza el concepto de una "fila" plana por un "**documento**" mucho más flexible.

- » Los documentos permiten datos embebidos (sub-documentos).
- » Soporte nativo para **arrays** (listas).
- » Se ajustan naturalmente a la forma en que los desarrolladores escriben código orientado a objetos moderno.
- » Estructuras anidadas reducen la necesidad de JOINS complejos.



Características Clave de MongoDB

Esquemas Flexibles

MongoDB no impone esquemas predefinidos a nivel de base de datos.

Esto significa que las claves y los valores de un documento no necesitan ser de tipos o tamaños fijos. Diferentes documentos en la misma colección pueden tener estructuras distintas, permitiendo a los desarrolladores iterar y adaptar la aplicación más rápido.

Escalabilidad Automática

MongoDB fue diseñado desde cero para escalar horizontalmente (scale out).

La base de datos maneja automáticamente el balanceo de los datos y el enrutamiento de las peticiones de los usuarios a través del clúster, sin intervención manual pesada.

Paso 1: Descargar el instalador

Community Edition

Overview

Features

Getting Started

Resources 

MongoDB Community Edition

The source-available, free-to-use version of MongoDB's document database. Ideal for learning, experimenting, and building applications in a self-managed environment.

Download Free

Explore fully managed MongoDB



Seleccioná la versión apropiada

MongoDB Download Center

El primer paso es obtener el binario oficial para Windows desde la página web de [MongoDB](https://www.mongodb.com).

Asegúrese de seleccionar los parámetros correctos para obtener el instalador Windows MSI.

- **Versión:** Seleccionar versión 7.0 (Community Edition).
- **Plataforma:** Seleccionar Windows (soporta Server 2022, 2019, Windows 11 en 64-bits).
- **Paquete:** Seleccionar "msi".
- Hacer clic en **Download** y ejecutar el archivo descargado.

The screenshot shows the MongoDB Download Center interface. It features three dropdown menus for selecting the version, platform, and package. The version is set to 8.2.6 (current), the platform is Windows x64, and the package is msi. Below the dropdowns are three buttons: a green 'Download' button with a download icon, a 'Copy link' button with a copy icon, and a 'More Options' button with a three-dot menu icon.

Version	8.2.6 (current)	▼
Platform	Windows x64	▼
Package	msi	▼

[Download](#)  [Copy link](#) [More Options](#) 

Paso 2: Ejecución y Configuración del Servicio



Tipo de Setup

En el asistente de instalación, seleccione **Complete** (recomendado). Esto instalará MongoDB y las herramientas en la ubicación predeterminada sin complicaciones adicionales.



Paso 2: Ejecución y Configuración del Servicio



Service Configuration

Seleccione "**Install MongoDB as a Service**". Permite que MongoDB inicie automáticamente en segundo plano. Utilice la opción "Run the service as Network Service user" (Por defecto).



Directorios

El instalador configurará el **Data Directory** (corresponde a `dbpath`) y el **Log Directory** (corresponde a `logpath`). Si no existen, el instalador los creará automáticamente.

Paso 3: Compass y el MongoDB Shell

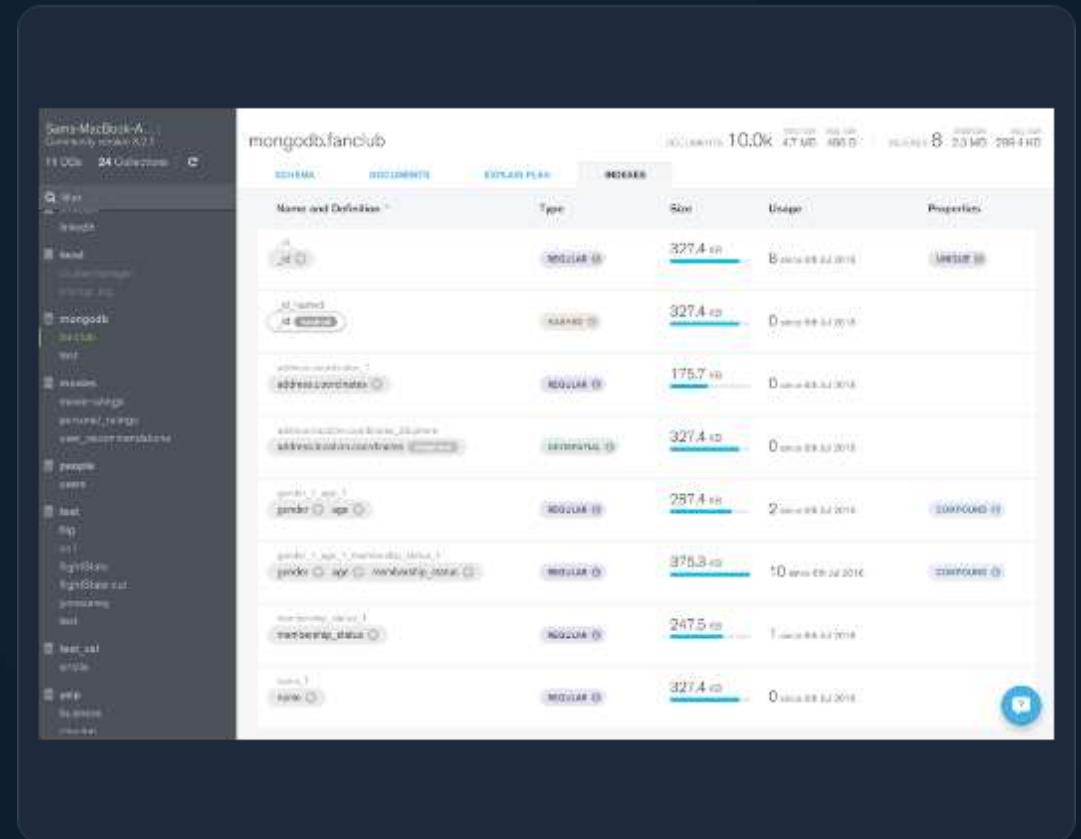
Herramientas Adicionales

MongoDB Compass: Durante el instalador MSI, marque la opción "Install MongoDB Compass". Es la Interfaz Gráfica de Usuario (GUI) oficial, excelente para explorar y manipular datos visualmente.

Instalar mongosh: IMPORTANTE. El shell moderno de MongoDB (`mongosh`) ya no viene incluido por defecto en el MSI del servidor.

» Debe descargar el zip de mongosh por separado.

» Extraer los binarios y agregar la ruta de `mongosh.exe` a su variable de entorno PATH de Windows.



Paso 4: Ejecución Manual desde CMD

Si no instaló como servicio

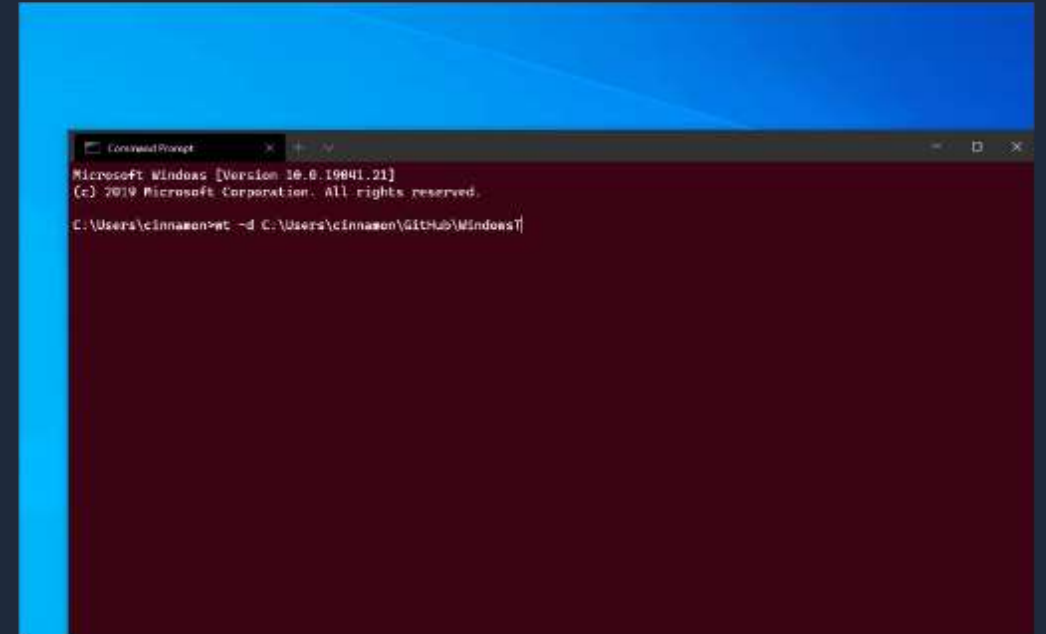
Abra el Intérprete de Comandos de Windows (`cmd.exe`) como **Administrador**.

```
C:\> md "\\data\db"
```

Este es el directorio por defecto requerido por MongoDB si no se especifica otro.

Para iniciar el servidor, ejecute el binario `mongod.exe` y apunte al directorio creado:

```
"C:\Program  
Files\MongoDB\Server\7.0\bin\mongod.exe" --  
dbpath="c:\data\db"
```





1. Verificar los Logs de Inicio

Si el servidor de base de datos está funcionando correctamente, la consola mostrará el mensaje: [initandlisten] waiting for connections. El servidor escucha por defecto en el puerto 27017.



2. Configuración del Firewall

Windows Defender puede mostrar una alerta de seguridad. Para solucionarlo, seleccione "Redes privadas" y haga clic en "Permitir acceso" para que `mongod.exe` pueda comunicarse en la red.



3. Conectar el Cliente

Abra una nueva ventana del Intérprete de Comandos y simplemente escriba mongosh.exe para conectarse al servidor local de MongoDB y comenzar a lanzar comandos.

Revisar los pasos de este showcase y descargar, instalar y configurar de forma exitosa tanto el servidor MongoDB como el JavaScript Shell en sus propios entornos de desarrollo locales.